

Context Engineering & Context Architecture

A field guide for designers building with AI coding assistants

1. The core reframe

You already design scarce space. A screen has only so much room; everything on it competes for attention, and cramming in more usually makes the experience worse, not better.

A model's **context window** works the same way. It's the finite space holding everything the model can “see” at the moment it responds — and just like a screen, relevance beats volume. Irrelevant material doesn't sit there harmlessly; it dilutes the signal and drags the output down.

The anchor idea: Deciding what goes into that scarce space is a real design act. It has a name — two, actually.

2. Two terms, one distinction

These are two sides of the same craft, and you already know the shape of it from your own work.

Context engineering is tactical and per-task: *what does the model see right now, for this one request?* You assemble it fresh each time.

Context architecture is structural and durable: *how is the whole information environment organized so the right things can be surfaced reliably, over and over?*

	Context engineering	Context architecture
What it is	What the model sees right now	How the information environment is structured
Timescale	Per request — dynamic	Persistent — set up once
Design analogy	Interaction design	Information architecture
You're deciding	What to put in front of the model <i>this time</i>	What exists to be drawn from, and how it's organized

The analogy is the shortcut: **engineering is to architecture as interaction design is to IA.** One is the moment; the other is the structure that makes good moments possible.

3. Where context comes from

Everything the model sees arrives from somewhere. Some sources are **static** — set up once and persist (the architecture side). Others are **dynamic** — assembled fresh for each request (the engineering side). The main pieces converge into the window:

Sources of context

Static · set once, persists

System prompt

Rules, role, format

Persistent memory

CLAUDE.md, projects

Dynamic · assembled per request

User input

Message, uploads

Conversation history

Prior turns

Retrieved knowledge

Search, RAG

Tool outputs

Results, status

Context window

Assembled per request

Finite, scarce space —
relevance beats volume

Model

Reads the window

Output

4. Reference — what counts as “context”

A fuller inventory of what can occupy that window, grouped by source. The static/dynamic tag maps back to architecture vs. engineering.

Source group	Items	Type
Instructions & config	System prompt; developer/app instructions; tool definitions; output-format specs; user preferences	Mostly static
Persistent knowledge	Memory across sessions; project files / knowledge base (CLAUDE.md); style/voice config	Static
Retrieved / grounded	Document chunks (RAG); web search results; database/API query results	Dynamic
Conversation	Prior user + assistant turns; summarized older history	Dynamic
Current input	The user's message; uploaded files; images/screenshots; pasted text or code	Dynamic
Tool & agent execution	Tool outputs; tool errors/status; intermediate reasoning or sub-agent results	Dynamic
Environment & metadata	Current date/time; user location; session/device metadata	Dynamic

5. Example 1 — Claude Code: *you* are the context engineer

When you work in Claude Code, you pull the levers yourself. Every choice about what the assistant can see is a context decision:

- `CLAUDE.md` **is context architecture you can hold in your hand** — a file that persists across every session, carrying the durable facts about your project (stack, conventions, structure). Set up once, drawn from every time.
- **Which files you reference in a given prompt is context engineering** — you decide, for *this* task, what the model needs in front of it.
- **Clearing or compacting the thread is context management** — when history stops helping and starts crowding, you reset the scarce space.

The lesson: a persistent context file is architecture; per-task file references are engineering. In Claude Code you're doing both by hand — which is the best way to feel how the levers work.

6. Example 2 — Claude.ai: the *product* is the context engineer

Now flip the vantage point. In Claude.ai — or any chat product — the app does the assembly for you, every time you hit enter. This is exactly the kind of system you're about to build.

- **The model is stateless.** It remembers nothing between requests, so the product reassembles context on every turn: your new message, plus the relevant history, plus anything retrieved.
- **Memory and Projects are architecture** — persistent context the product maintains so it can surface the right things later.
- **Uploaded files get chunked and pulled from, not dumped in whole; old turns get summarized or dropped.** That's the product doing engineering on your behalf, silently.

The lesson: a finished AI product *is* a context system. What a user experiences as “it just knows” is a designed pipeline assembling the right context at the right moment. That pipeline is now part of your job.

7. Considerations you'll hit

You don't need the mechanics yet — but you'll meet these words, so know what they point at. We go deeper on them in the evaluation and scaling sessions.

- **Token limits** — the window is finite, measured in tokens. Everything competes for a fixed budget.
- **Chunking** — big documents get split into pieces so only the relevant parts get pulled in.
- **Retrieval (RAG)** — fetching relevant pieces at request time instead of stuffing everything in up front.

- **Context rot** — as the window fills with marginal material, output quality drops. More is not better.

8. Why this matters for designers

This is *behavior before interface*, made concrete. What a system knows — and *when* it knows it — shapes how it behaves long before any pixel is placed.

Deciding what context a system has access to isn't a technical detail you hand off. It's an upstream design decision that determines whether the thing feels sharp or clueless. Learn to shape the information environment, and you're designing the behavior itself.